



C++ Coding files presentation

Maximillien and Siddesh



What was the Assignment

Radiation Protection (RP) Simulation Project

- **Objective:** Develop a high-performance C++ simulation to track radioactive particles (neutrons and protons) as they interact with shielding and target regions.
- **The Model:** Implement a time-step loop ($t = N \Delta t$) to calculate particle movement and spatial containment within 3D geometries.
- **Mandatory Features:** Build realistic physics, including a bounding box to kill out-of-bounds particles, velocity noise for emission, and secondary particle generation (scattering/absorption).
- **Advanced Optimization:** Refactor the code using Object-Oriented principles (like the parent `Box` class) and visualize results using ParaView to verify simulation accuracy.

CI & CD Tools

To improve our development efficiency we used **Doctest**, **Doxygen** and **github actions**

- **Doctest**
 - Run test to ensure the code is bug free.
 - We created ***make test*** command to easily run tests
 - Inside **test.cpp**, we created over 168 test cases for ensuring the quality of our implementation.
- **Doxygen**
 - Doxygen is a library that creat automatic documentation from comments in the code.
 - Similarly we the commend ***make gen-doc*** will automatically create the documentation
- **Github actions**
 - We created a github issue template for bug report.
 - Those issues name are automatically formatted by a github action.
- **Git repository**
 - for version control we made a git repository

Box code

Why it was added

- **Code Refactoring & Efficiency:** In the original project structure, the Shielding and ROI (Region of Interest) classes both used identical logic for their contains() methods. Creating a parent class eliminates this redundancy.
- **Object-Oriented Best Practices:** By implementing a base Box class, I utilized **inheritance**, a core principle of C++. This allows common properties (like 3D coordinates) and behaviors (like checking if a particle is inside) to be managed in one place.
- **Scalability:** This structure makes it much easier to add future geometric objects—such as a BoundingBox or a Source area—without rewriting the spatial logic for each.

What It Changed

- **Simplified Logic:** The contains() method now lives in the Box class. It uses a single boolean check to determine if a particle's (x, y, z) coordinates fall within the defined xmin/xmax, ymin/ymax, and zmin/zmax boundaries.
- **Inheritance Hierarchy:**
 - **Base Class:** Box (stores the 3D bounds and the contains() logic).
 - **Derived Classes:** Shielding and ROI now inherit from Box, making the code cleaner and more professional.
- **Maintenance:** If the geometry logic needs to change (e.g., shifting from a 3D box to a different shape), I only need to update the base class or create a new derived class rather than modifying every individual component of the simulation.

Vector 3

- **Code Readability:** Instead of passing around three separate variables (x, y, z) for every function, I grouped them into a single Vector3 struct. This makes the function signatures much shorter and easier to read.
- **Data Organization:** A Vector3 better represents the physical reality of a "point" or "velocity" in 3D space as a single entity rather than three unrelated numbers.
- **Reduced Complexity:** It simplifies mathematical operations—like updating positions or calculating distance—by allowing me to work with one object instead of three individual coordinate updates.

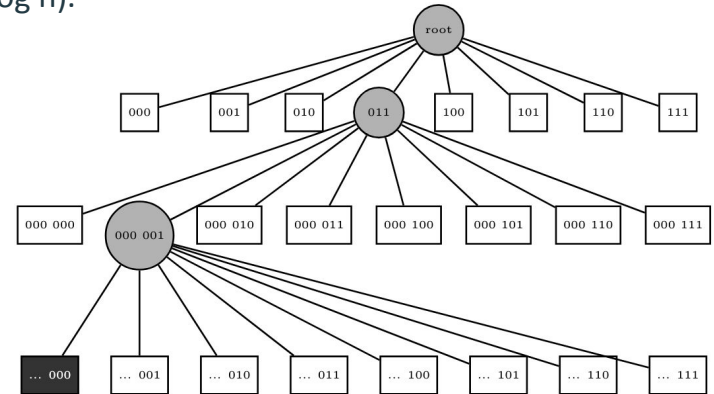
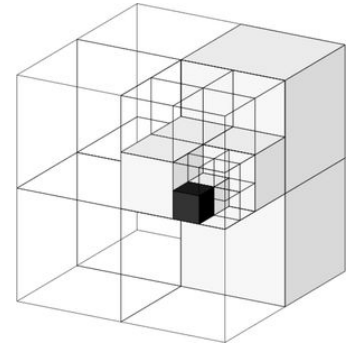
Barnes-Hut Algorithm Implementation

Purpose:

- We wanted to add the effects of electromagnetic interaction to the simulation.
- This make the simulation a n-body system that can be approximated with the Barnes-Hut Algorithm that make spatial subdivision with a octree (a tree with at most 8 child).
- Using Barnes-Hut change the complexity from $O(n^2)$ to $O(n \log n)$.

Our implementation:

- We implemented octree that feature lazy evaluation, meaning that the vertices and forces in the octree are only calculated when needed.
- The root box is the simulation bounding box.
- To calculate force some attribute has been added to the particle class, such as mass and acceleration.



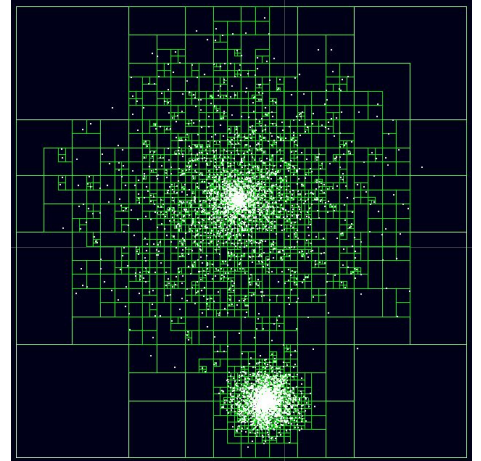
Barnes-Hut Algorithm explanation

1. Construct the Octree

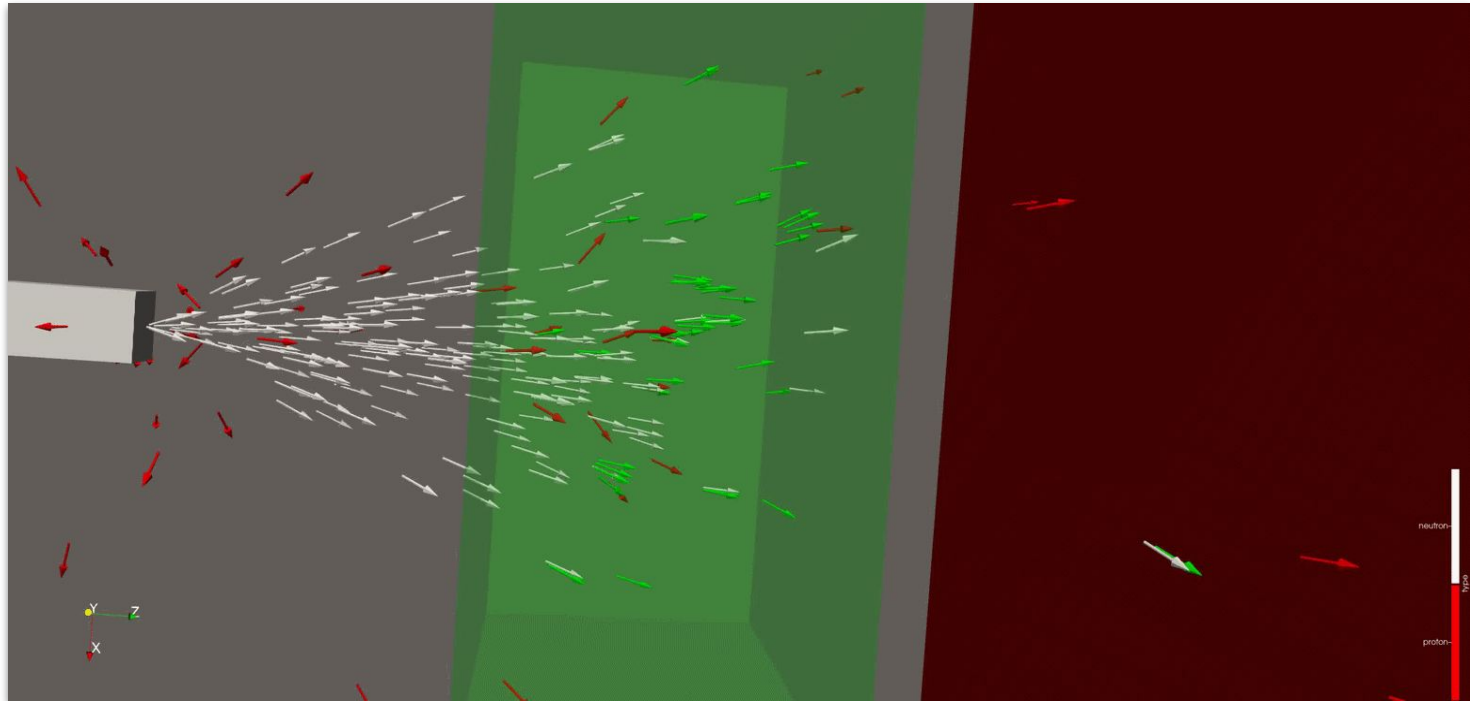
- Make a octree such that each leaf has exactly one particle.
- All living particle must be in the octree.

2. Estimate N-Body Forces & Calculate Centers of Charge

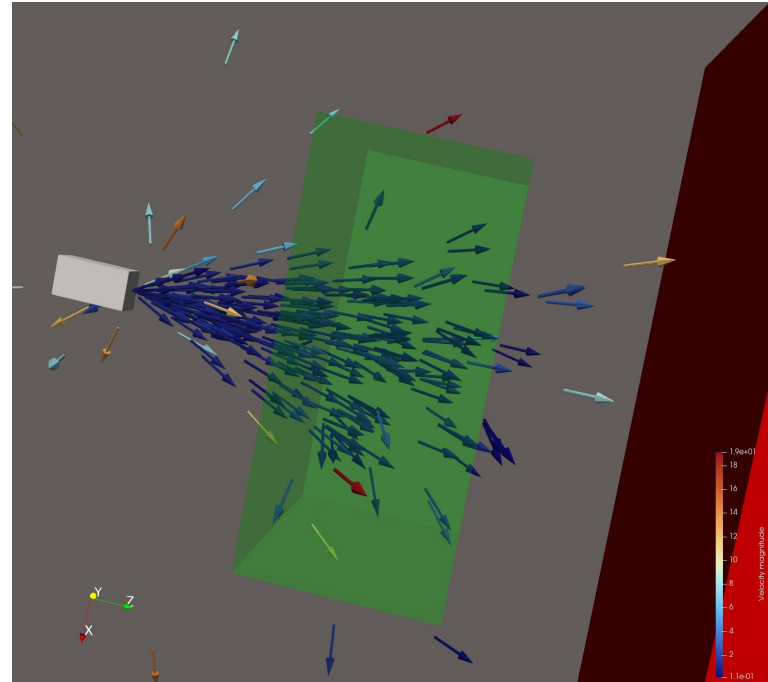
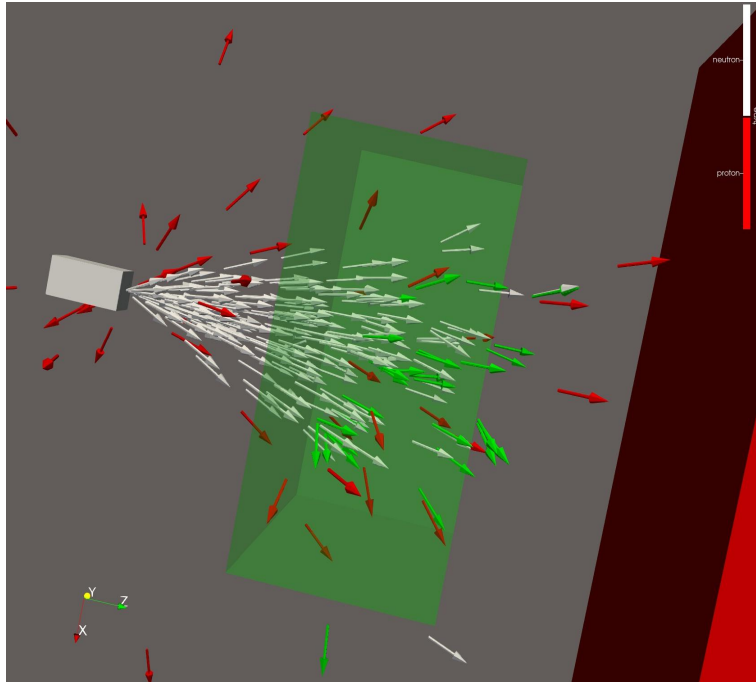
- The barnes-hut criteria is define as $s / d < \theta$ where s is a cube length, d the distance to the other particle and θ the approximation precision.
- If the barnes-hut criteria is satisfied for a cube,we approximate its child particle as one particle by calculating the center of charge (equivalent to the center of mass).
- We then calculate the coulomb force magnitude between this approximated particle and the current particle.
- Repeat this process for every living particle.



Results - Visual glimpse



Results - Particle Type vs Particle Velocity



Further Improvement

- Making a monte-carlo simulation to estimate expected value of irradiation of the ROI.
- Adding the possibility to have an electrically charged radiation shield.
- Adding neutron moderation in as shielding.

Thank You